

NTM Workshop

University of Queensland, Australia

Day 1: Introduction to R (14:30 – 16:30)

Today's goals:

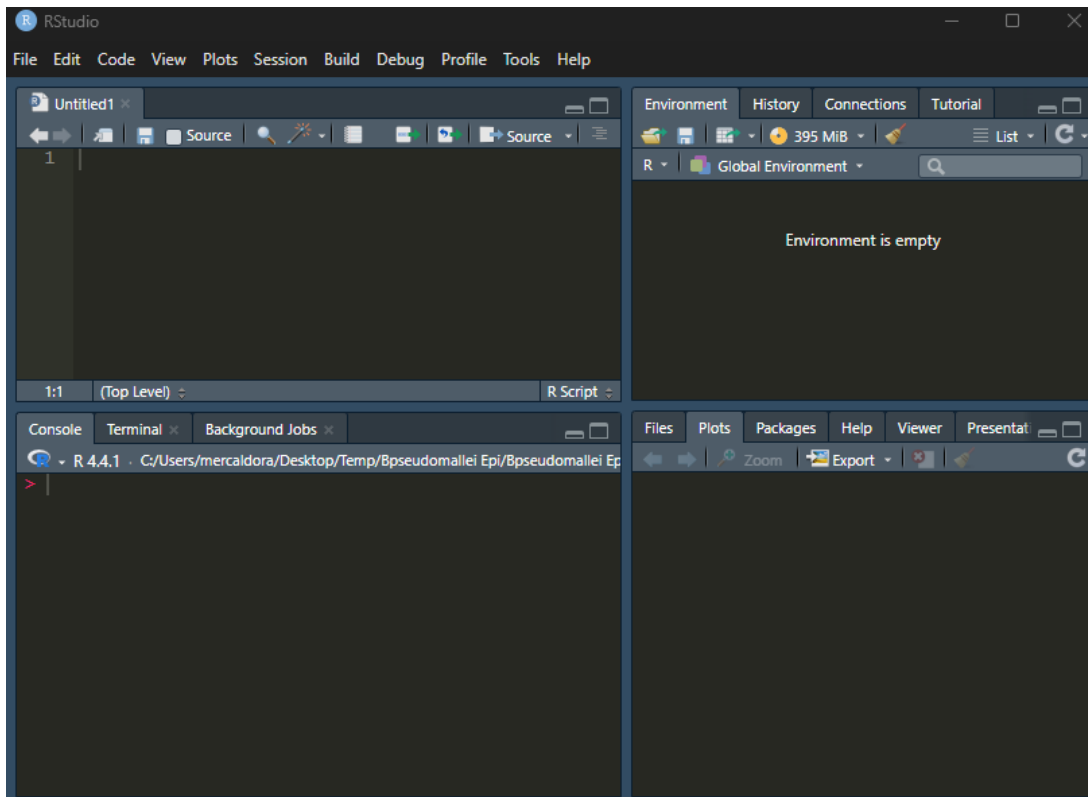
Become familiar with RStudio, basic R commands, data imports and manipulation.

Tasks:



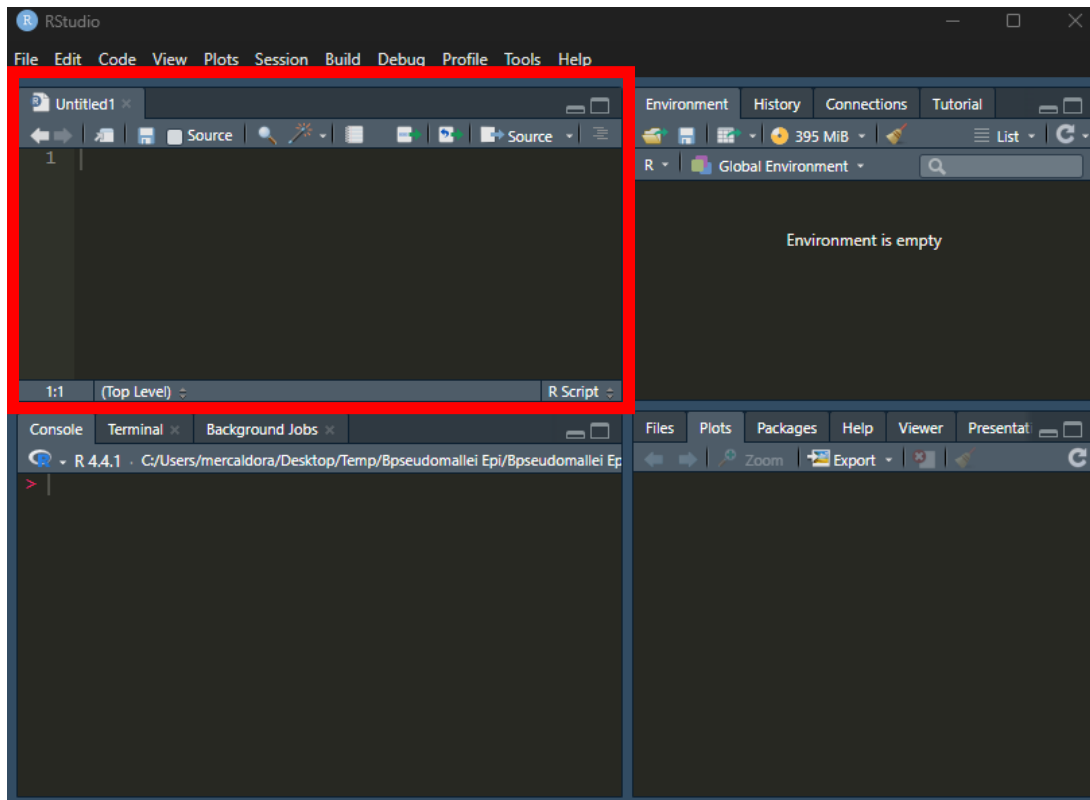
Open RStudio by clicking on the icon or by running the program from Start or the program directory of your computer.

Depending on how you installed R, it may be in light mode or dark mode (white background vs black background). The tutorials for this workshop will use Rachel's computer, which has R installed with dark mode.



In the top left of the screen is the source code window. This is where you can write code or give R commands *without running them right away*. The source file can be saved so you can come back to it later. This is where you will be doing most of your work in R.

For now, the source file is called “untitled1,” because it is not saved.



1. You can save the source code file to your computer if you choose. You can save it to your Desktop, or to any folder. It is not necessary to save it for the workshop.

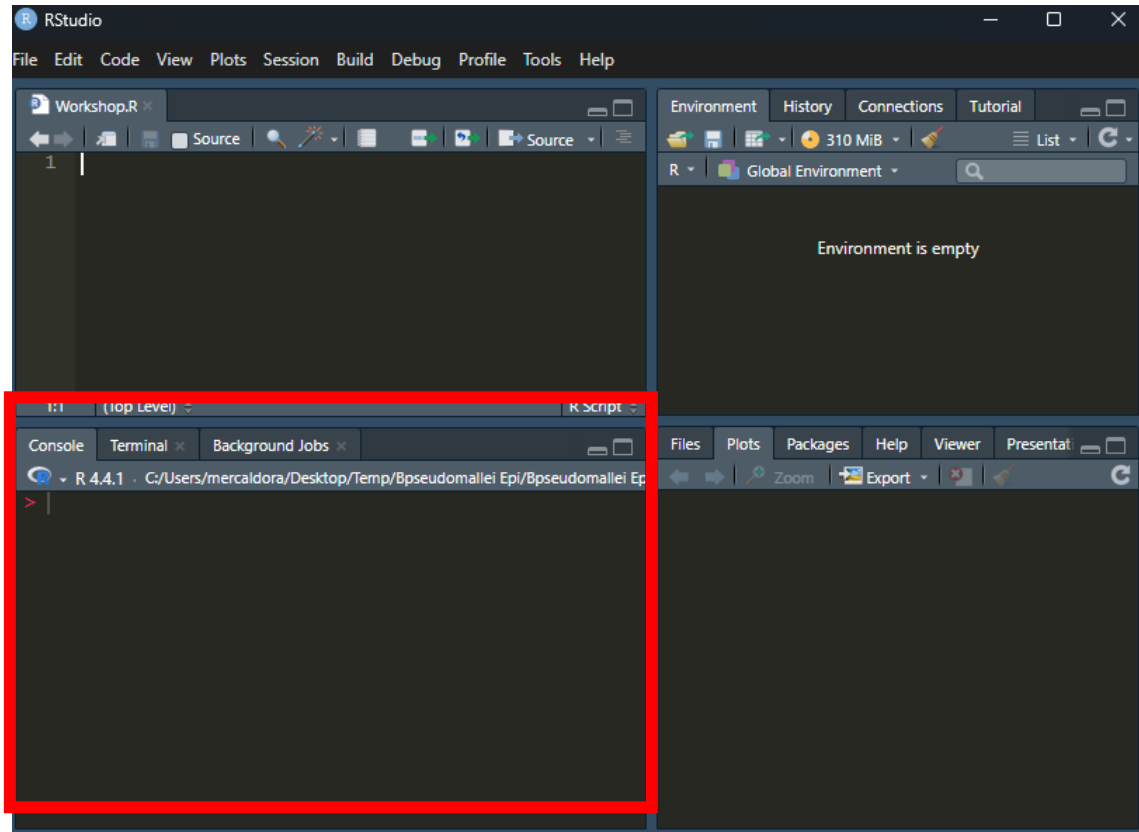
Save the source file by navigating through the File option at the very top left of the screen.

Recommendation: create a workshop folder that is easy to find. You can save the source code file in that folder, as well as any other files we will use for the rest of the workshop.

R source code files are saved with a .R file extension. If you click on the file, it should automatically open in RStudio, even when RStudio is not already running on your computer.

The bottom left pane is the Console. This is where the code you execute is run and the results appear.

You can either type directly into the Console, or you can run lines of code from the source file above.



Let's run some code from both the Console and the Source File

2. The most basic use of R is as a calculator. In the Console (bottom left), type the following statements at the R prompt (“>”) and then press your “enter” or “return” key.

```
3  
3 + 5  
3.4 + 6  
x = (3.4 + 6)/4
```

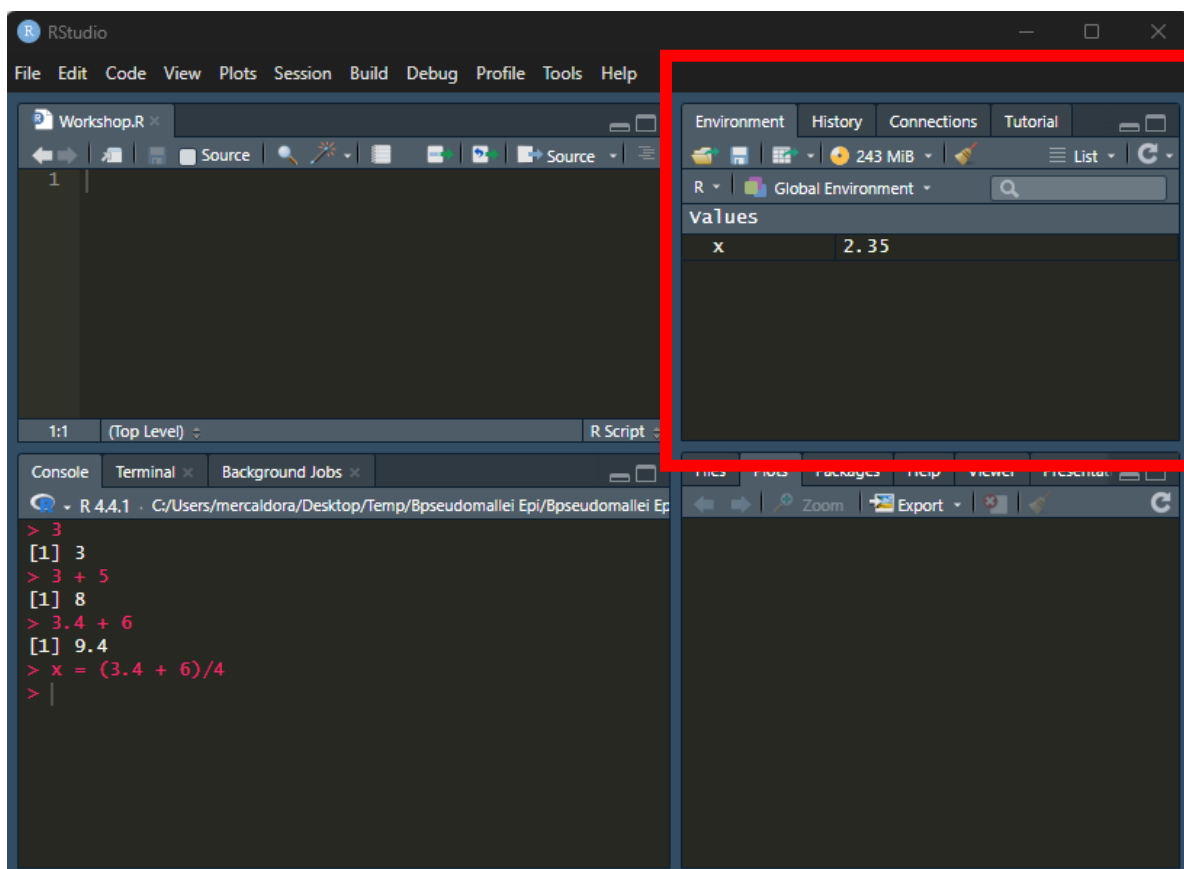
Your console should look something like this:

```
> 3  
[1] 3  
> 3 + 5  
[1] 8  
> 3.4 + 6  
[1] 9.4  
> x = (3.4 + 6)/4
```

You may have noticed that nothing was returned when you ran $x = (3.4 + 6)/4$. This is because R uses the `=` symbol for assignment.

Assignment is when a value or object is assigned to a variable. In this case, the variable “x” was assigned the value of $(3.4 + 6)/4$.

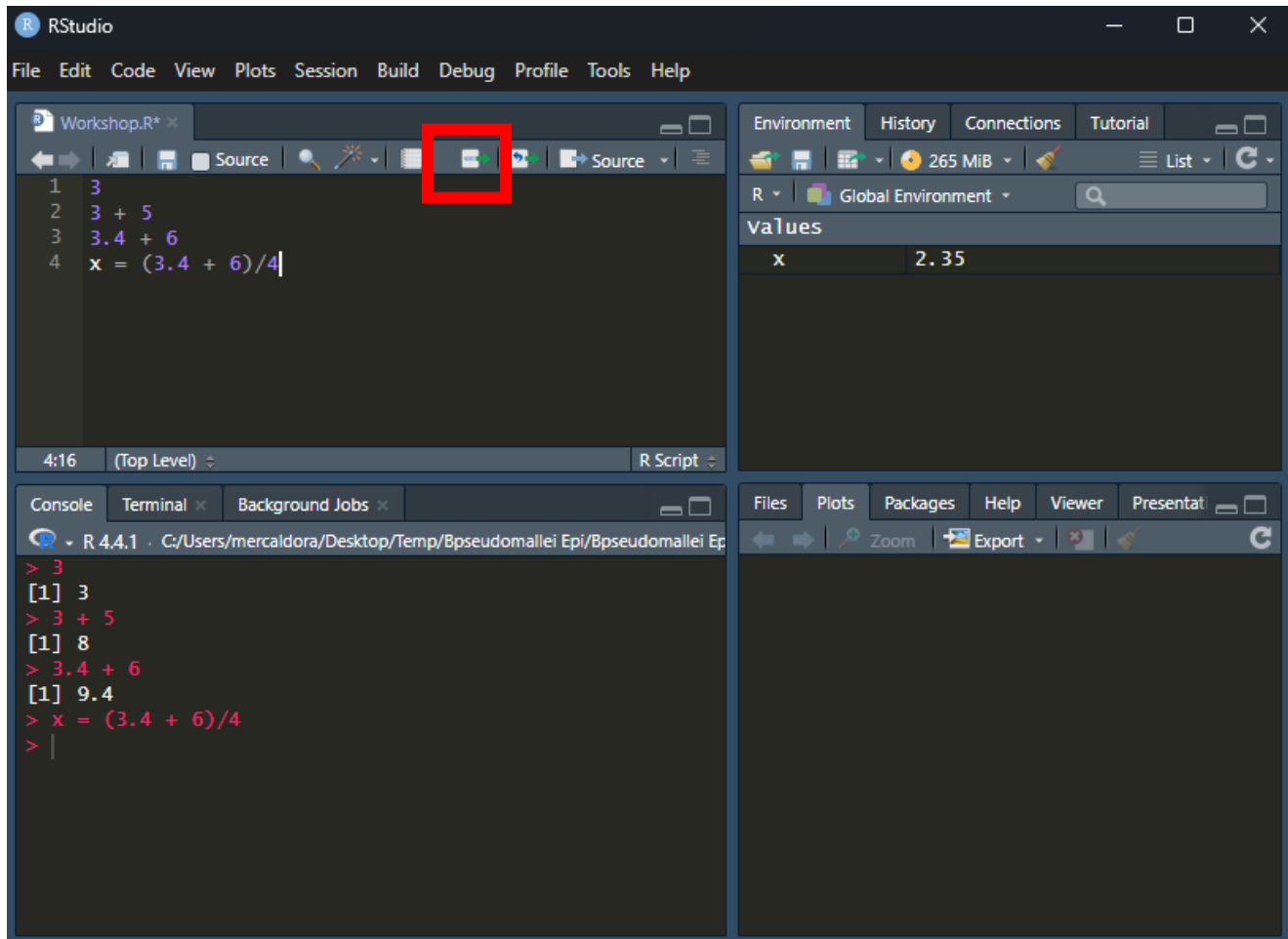
In R, all variables are stored in the Environment:



The Environment is where R stores all the data, variables, and objects you have asked it to remember. By using the “=” symbol, you have asked R to remember that x is $(3.4 + 6)/4$, or 2.35.

You can use the new “x” in your code, now. You can also ask R to print the value of x by simply typing x in your console and hitting “return” or “enter” on your keyboard. Try it.

- Let’s try the same exercise, but instead of typing the commands into the Console directly, type them into the Source File, in the top left. Each statement should be on its own row. You do not need the `>` prompt symbol when you are using the Source file pane. When you are done typing, you can highlight all the rows at once to run them, or select one to run at a time. To run, make sure you have either highlighted all the rows you want to run, or that your cursor is in a single row that you want to run. Then, hit the “run” icon.



You should see the Console prints the same results, regardless of if the commands come from the Console or the Source file.

4. Give `x` a new value, such as 8, or `sin(2)`, or `pi`. You can do this in either the Console or the Source file.

If the “=” symbol assigns values to variables, how do we determine if something is equal? In R, we use double equal symbols when we want to use “equals.”

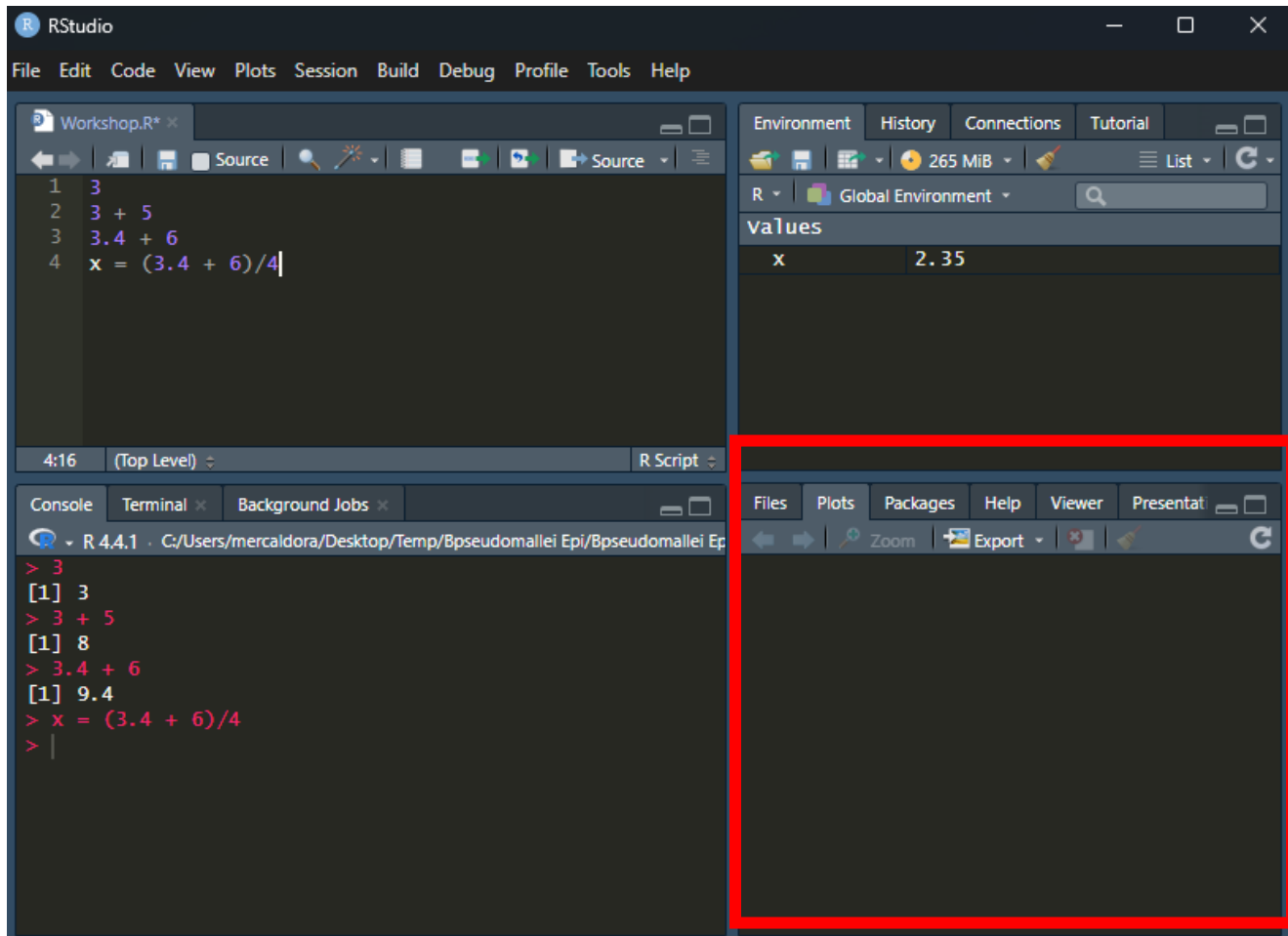
5. Try `2 == 3` and see what the Console prints. Then, try `2 == 2`.

The “#” symbol tells R *not* to execute whatever follows on that row. We usually use the # for comments or to “hide” code from R.

6. Try running `#2 + 3`. What happens? Try `2 + 3 #this is an example`

Next to the Environment tab in the top right pane is the “History” tab. Select it. You will see that this holds the history of the statements and commands you have sent to the Console, either by directly typing them into the Console or by running them from the Source File.

The final pane is the bottom right, and there are two primary tabs we are interested in.

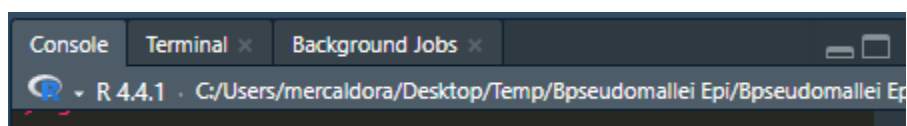


In the screenshot above, “Plots” is selected. Here, any plots or graphs we create in R can be viewed.

The other primary option is “Files.” Here, we will see the files R has access to.

Working directory

The files that R can access depend on the working directory R is in. In the screenshots shared here, I have opened R from a specific project directory, related to *Burkholderia pseudomallei* epidemiology. You can see my working directory just above the Console:



It is best to set your working directory to wherever your data files are stored. For example, if you choose to save the data for this workshop in the folder called “Workshop” that you created on your Desktop, you can set the working directory to Desktop/Workshop via the following steps:

In the top left of the program, you will see the banner options for “File,” “Edit,” etc. One of those options is “Session.”

Select “Session”

Select “Set Working Directory” and then “Choose directory.”

A File Explorer window will appear and you can navigate to the folder where the data are stored.

If you know the path to the folder with your data, you can also type the command to change the directory into the Console or include it in your Source File. For example, my folder:

```
> setwd("C:/users/mercaldora/Desktop/Workshop")
```

Packages

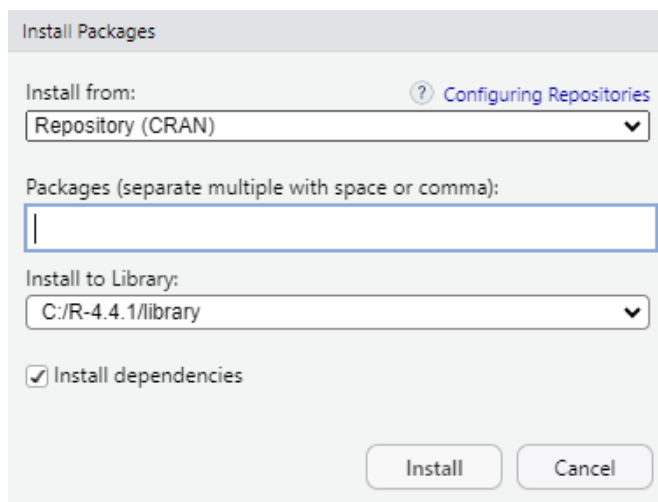
R has a lot of great tools built into the program, but not everything is available right away. Instead, users have to install packages, which are collections of commands and functions that other programmers have made available for use.

We will use several packages for our analysis. Let’s get them installed:

At the top of the screen, near the Session option, you should see a “Tools” option.

Select “Tools,” then “Install packages...”

A new window will pop open:



First, make sure the checkbox for “Install dependencies” is checked. Then, type the names of the following packages, separated by a comma (“,”), into the Packages box. You can leave the other boxes as they are:

```
tidyverse, stringr, readxl, ggplot2, glmnet, zoo, data.table, lattice,  
MASS, boot, sf, ggpubr
```

You can also copy the list above and paste it in the Packages box.

Select “Install” and allow the program to run and install all the packages.

This can take quite some time and you will likely see a lot of text appear in the Console. Do not worry. This is normal.

When your Console shows the > prompt once more, R has finished installing the packages. At this point, they are not ready for use. At the beginning of Source files and saved/shared code, you will often find a number of rows of commands that are calling the function “library” on different package names. This command makes the package available for this session of R. Whenever you open R, you will have to run the library() command for the packages you want to use.

7. Use the library() command to load the newly-installed packages for your R session. For example:

```
library("tidyverse")
```

You will need a separate row and command for each package.

*Congratulations. You have completed the first part of the Introduction to R workshop. Now, please **close R** and then open the “Day-1.Rmd” file that you were also sent.*